

SESSION 14 : Window Functions - Part 2

TASKS

1. Create a table called Orders with columns: order_id, user_id, order_date, and total_amount. Insert at least 7 sample rows representing different users and dates, similar to how food orders appear in Zomato or Swiggy.

Query / Answer:

```
CREATE TABLE Orderss (  
    order_id INT PRIMARY KEY,  
    user_id INT,  
    order_date DATE,  
    total_amount DECIMAL(10,2)  
);
```

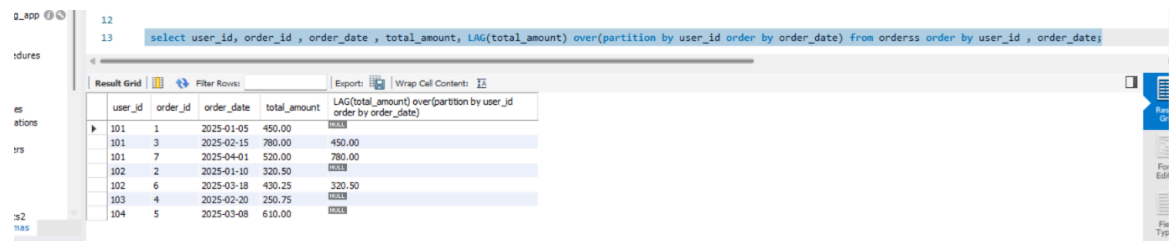
Image / Screenshot:

2. Write a SQL query using the LAG() function to show each user's order_id, order_date, and the total_amount of their previous order (if any), ordered by user and date.

Query / Answer:

```
select user_id, order_id , order_date , total_amount, LAG(total_amount) over(partition by user_id order by order_date) from orders order by user_id , order_date;
```

Image / Screenshot:



The screenshot shows a SQL query editor with the following query:

```
select user_id, order_id , order_date , total_amount, LAG(total_amount) over(partition by user_id order by order_date) from orders order by user_id , order_date;
```

The results are displayed in a grid with the following columns: user_id, order_id, order_date, total_amount, and LAG(total_amount) over(partition by user_id order by order_date). The data is ordered by user_id and then by order_date.

user_id	order_id	order_date	total_amount	LAG(total_amount) over(partition by user_id order by order_date)
101	1	2025-01-05	450.00	NULL
101	3	2025-02-15	780.00	450.00
101	7	2025-04-01	520.00	780.00
102	2	2025-01-10	320.50	NULL
102	6	2025-03-18	430.25	320.50
103	4	2025-02-20	250.75	NULL
104	5	2025-03-08	610.00	NULL

3. Using the same Orders table, write a SQL query with the LEAD() function to display each order_id, order_date, and the next order's total_amount for the same user.

Query / Answer:

```
select user_id , order_id , order_date, total_amount , LEAD(total_amount) over(partition by user_id order by order_date) from orders order by user_id , order_date;
```

Image / Screenshot:

The screenshot shows a SQL IDE interface. The query editor at the top contains the following SQL query:

```
select user_id , order_id , order_date, total_amount , LEAD(total_amount) over(partition by user_id order by order_date) from orders order by user_id , order_date;
```

Below the query editor, the 'Result Grid' tab is active, displaying the results of the query. The results are as follows:

user_id	order_id	order_date	total_amount	LEAD(total_amount) over(partition by user_id order by order_date)
101	1	2025-01-05	450.00	780.00
101	3	2025-02-15	780.00	520.00
101	7	2025-04-01	520.00	430.25
102	2	2025-01-10	320.50	430.25
102	6	2025-03-18	430.25	250.75
103	4	2025-02-20	250.75	610.00
104	5	2025-03-08	610.00	

4. Write a SQL query to calculate the running total of total_amount for each user, showing order_id, order_date, total_amount, and a column running_total that accumulates the sum as you move through each user's orders.

Query / Answer:

```
select user_id , order_id , order_Date , total_amount , sum(total_amount) over(partition
by user_id order by order_date) as runningtotal from orderss order by user_id ,
order_date;
```

Image / Screenshot:

The screenshot shows a SQL IDE interface. The query editor contains the following SQL query:

```
select user_id , order_id , order_date, total_amount , LEAD(total_amount) over(partition by user_id order by order_date) from orderss order by user_id , order_date;
```

Below the query editor, the results are displayed in a table with the following columns: user_id, order_id, order_date, total_amount, and runningtotal. The table contains 8 rows of data.

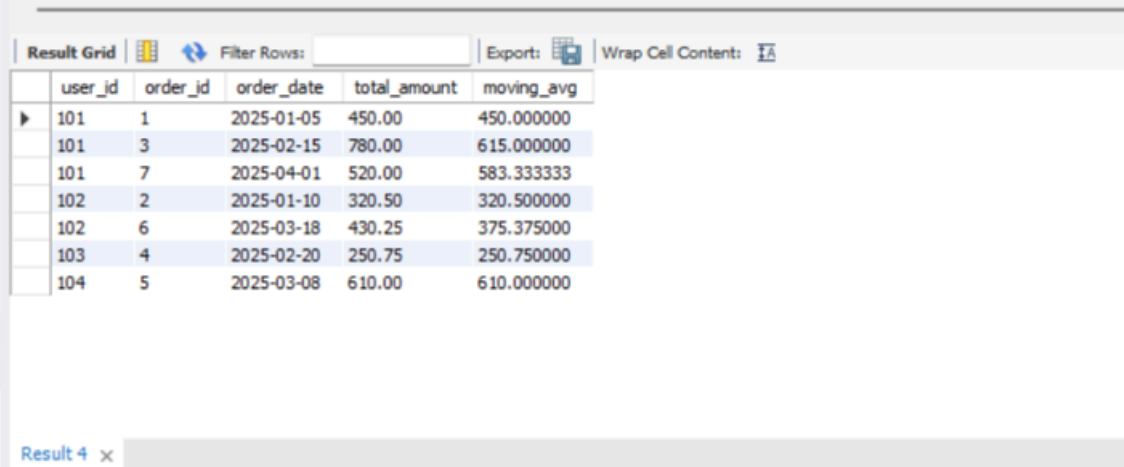
user_id	order_id	order_date	total_amount	runningtotal
101	1	2025-01-05	450.00	450.00
101	3	2025-02-15	780.00	1230.00
101	7	2025-04-01	520.00	1750.00
102	2	2025-01-10	320.50	320.50
102	6	2025-03-18	430.25	750.75
103	4	2025-02-20	250.75	250.75
104	5	2025-03-08	610.00	610.00

5. Write a SQL query to calculate a 3-order moving average of total_amount for each user, showing order_id, order_date, total_amount, and moving_avg columns.

Query / Answer: (ChatGPT need to ask)

```
SELECT
  user_id,
  order_id,
  order_date,
  total_amount,
  AVG(total_amount) OVER (
    PARTITION BY user_id
    ORDER BY order_date
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
  ) AS moving_avg
FROM Orders
ORDER BY user_id, order_date;
```

Image / Screenshot:



The screenshot shows a database query result grid with the following data:

	user_id	order_id	order_date	total_amount	moving_avg
▶	101	1	2025-01-05	450.00	450.000000
	101	3	2025-02-15	780.00	615.000000
	101	7	2025-04-01	520.00	583.333333
	102	2	2025-01-10	320.50	320.500000
	102	6	2025-03-18	430.25	375.375000
	103	4	2025-02-20	250.75	250.750000
	104	5	2025-03-08	610.00	610.000000

Result 4 x